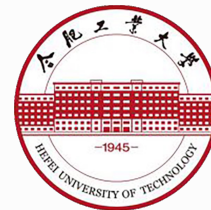




数据库系统

第六章 关系数据库理论

胡金鹏



本篇讲解在开发应用系统中如何在已经选定的数据库管理系统基础上设计数据库，如何基于数据系统编程。

第六章 关系理论

关系规范化理论

第七章 数据库设计

步骤与方法

第八章 数据库编程

SQL、ODBC



6.1 问题的提出

6.2 规范化

6.3 数据依赖的公理系统

6.4 模式的分解



关系模式由五部分组成，即一个五元组：

R(U, D, DOM, F)

R: 关系名

U: 组成该关系的属性名集合

D: 属性组U中属性所来自的域的集合

DOM: 属性向域的映像集合

F: 属性间数据的依赖关系集合

学 号	姓 名	年 龄	性 别	系 名	年 级
2015004	王小明	19	女	社会学	2015
2015006	黄大鹏	20	男	商品学	2015
2015008	张文斌	18	女	法律	2015
...	...	...	...	...	...

举例说明

✓学生（姓名，学号，年龄）

✓属性集合就是 姓名、学号、年龄、...等属性组成的一个属性组

✓属性来自的域：姓名来自的域：世界上存在的所有姓名的集合

✓Dom:属性向域的映像集合，比如{王小明，黄大鹏，张文斌}是姓名这个属性的一个Dom，



【说明】

- 由于D、DOM与模式设计关系不大，因此在本章中把关系模式看作一个三元组： $R(U, F)$
- 当且仅当U上的一个关系r满足F时，r称为关系模式 $R(U, F)$ 的一个关系
- 作为二维表，关系要符合一个最基本的条件：每个分量必须是不可分开的数据项，满足了这个条件的关系模式就属于**第一范式**
(1NF)



什么是数据依赖

- 一个关系内部属性与属性之间的约束关系，通过属性间值的相等与否体现出来的数据间相互联系
- 现实世界属性间相互联系的抽象
- 数据内在的性质
- **语义**的体现

现实世界属性间相互联系的抽象，属于数据内在的性质



【例】描述一个学生关系，可以有学号、姓名、系名等属性。

一个学号只对应一个学生，一个学生只在一个系中学习，“学号”确定后，学生的姓名及所在系的系名被唯一确定。

$Sname=f(Sno)$ ， $Sdept=f(Sno)$ 即Sno函数决定Sname，Sno函数决定Sdept，记作 $Sno \rightarrow Sname$ ， $Sno \rightarrow Sdept$



【例】建立一个描述学校教务的关系模式：

学生的学号 (Sno)、所在系 (Sdept)、系主任姓名 (Mname)、课程号 (Cno)、成绩 (Grade)

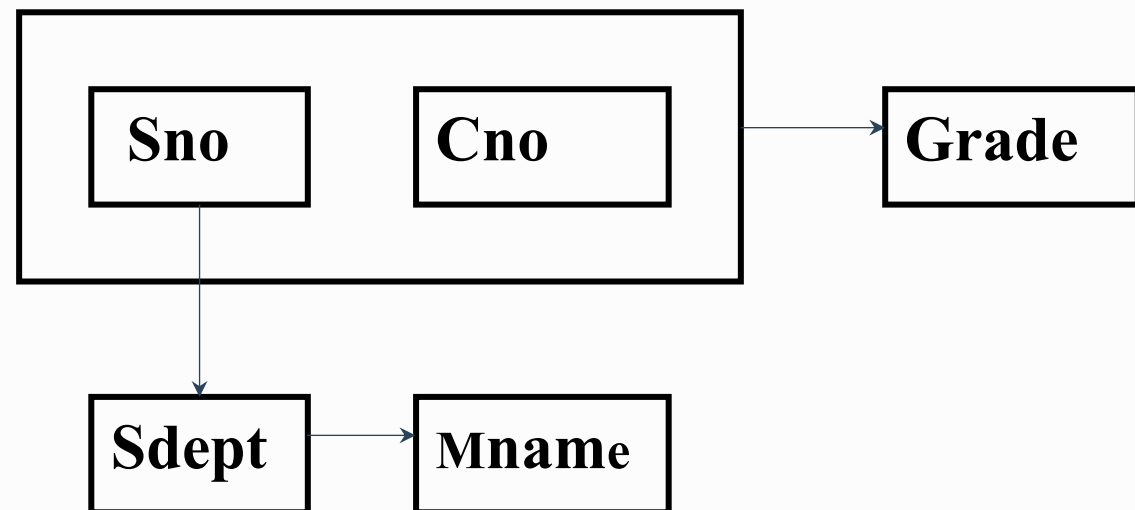
一个系有若干的学生，但一个学生只属于一个系；一个系只有一名负责人；一个学生可以选修多门课，每个课程有若干学生选修；每个学生学习每一门课有一个成绩。

单一的关系模式： Student (U, F)

$U = \{ Sno, Sdept, Mname, Cno, Grade \}$

属性组U上的一组函数依赖F：

$F = \{ Sno \rightarrow Sdept, Sdept \rightarrow Mname, (Sno, Cno) \rightarrow Grade \}$





数据依赖存在的问题

一、数据冗余太大

浪费大量的存储空间，每一个系主任的姓名重复出现，重复次数与该系所有学生的所有课程成绩出现次数相同。

Sno	Sdept	Mname	Cno	Grade
200201001	计算机系	李明	C1	95
200201002	计算机系	李明	C1	87
200201003	计算机系	李明	C1	67
200201004	计算机系	李明	C1	59
200201005	计算机系	李明	C1	90
.....				



数据依赖存在的问题

二、更新异常

数据冗余，更新数据时，维护数据完整性代价大。某系更换系主任后，必须修改与该系学生有关的每一个元组，否则会出现数据不一致的异常。

Sno	Sdept	Mname	Cno	Grade
200201001	计算机系	李明	C1	95
200201002	计算机系	李明	C1	87
200201003	计算机系	李明	C1	67
200201004	计算机系	李明	C1	59
200201005	计算机系	李明	C1	90
.....				



数据依赖存在的问题

三、插入异常

如果一个系刚成立，尚无学生，则无法把这个系及其系主任的信息存入数据库。

四、删除异常

如果某个系的学生毕业了，则在删除该系学生信息的同时把这个系及其系主任的信息页丢掉了。



数据依赖存在的问题

结论：

Student关系模式不是一个好的模式。

“好”的模式：

⇒ 无插入异常、删除异常、更新异常，数据冗余少

原因：

⇒ 由存在于模式中的某些数据依赖引起的

解决方法：⇒ 通过分解关系模式来消除其中不合适的数据依赖



把这个单一模式分成3个关系模式：

$S(Sno, Sdept, Sno \rightarrow Sdept);$

$SC(Sno, Cno, Grade, (Sno, Cno) \rightarrow Grade);$

$DEPT(Sdept, Mname, Sdept \rightarrow Mname)$

规范化理论正是用来改造关系模式，通过分解关系模式来消除其中不合适的

数据依赖，以解决插入异常、删除异常、更新异常和数据冗余问题。



关系数据库理论



6.1 问题的提出

6.2 规范化

6.3 数据依赖的公理系统

6.4 模式的分解



6.2.1 函数依赖

6.2.2 码

6.2.3 范式

6.2.4 2NF

6.2.5 3NF

6.2.6 BCNF

6.2.7 多值依赖

6.2.8 4NF

6.2.9 规范化小结



【定义】设 $R(U)$ 是一个属性集 U 上的关系模式， X 和 Y 是 U 的子集。

若对于 $R(U)$ 的任意一个可能的关系 r ， r 中不可能存在两个元组在 X 上的属性值相等，而在 Y 上的属性值不等，则称“ **X 函数确定 Y** ”或“ **Y 函数依赖于 X** ”，记作 $X \rightarrow Y$ 。

Sno	Sdept	Mname	Cno	Grade
200201001	计算机系	李明	C1	95
200201002	计算机系	李明	C1	87
.....				

1. 所有关系实例均要满足
2. 语义范畴的概念
3. 数据库设计者可以对现实世界作强制的规定



【例】 Student(Sno, Sname, Ssex, Sage, Sdept)

假设不允许重名，则有：

$Sno \rightarrow Ssex$, $Sno \rightarrow Sage$, $Sno \rightarrow Sdept$, $Sno \longleftrightarrow Sname$

$Sname \rightarrow Ssex$, $Sname \rightarrow Sage$, $Sname \rightarrow Sdept$

但 $Ssex \not\rightarrow Sage$, $Ssex \not\rightarrow Sdept$



在关系模式 $R(U)$ 中，对于 U 的子集 X 和 Y ，

(1) 若 $X \rightarrow Y$ ，但 $Y \not\subseteq X$ ，则称 $X \rightarrow Y$ 是**非平凡的函数依赖**

(2) 若 $X \rightarrow Y$ ，但 $Y \subseteq X$ ，则称 $X \rightarrow Y$ 是**平凡的函数依赖**(自然成立)

例：在关系 $SC(Sno, Cno, Grade)$ 中：

非平凡函数依赖： $(Sno, Cno) \rightarrow Grade$

平凡函数依赖： $(Sno, Cno) \rightarrow Sno$

$(Sno, Cno) \rightarrow Cno$

通常都是讨论**非平凡的函数依赖**



在关系模式 $R(U)$ 中，对于 U 的子集 X 和 Y ，

(3) 若 $X \rightarrow Y$ ，则 X 称为这个函数依赖的决定属性组，也称为**决定因素**

(4) 若 $X \rightarrow Y$ ，并且 $Y \rightarrow X$ ，则记为 $X \longleftrightarrow Y$

(5) 若 Y 不函数依赖于 X ，则记为 $X \not\rightarrow Y$



【定义】在 $R(U)$ 中，如果 $X \rightarrow Y$ ，并且对于 X 的任何一个真子集 X' ，都有 $X' \not\rightarrow Y$ ，则称 Y 对 X **完全函数依赖**，记作 $X \xrightarrow{F} Y$ 。

若 $X \rightarrow Y$ ，但 Y 不完全函数依赖于 X ，则称 Y 对 X **部分函数依赖**，记作 $X \xrightarrow{P} Y$ 。

例：单一的关系模式： Student (U, F)

$U = \{ \text{Sno}, \text{Sdept}, \text{Mname}, \text{Cno}, \text{Grade} \}$

$(\text{Sno}, \text{Cno}) \xrightarrow{F} \text{Grade}$ 是完全函数依赖，

$(\text{Sno}, \text{Cno}) \xrightarrow{P} \text{Sdept}$ 是部分函数依赖

因为 $\text{Sno} \rightarrow \text{Sdept}$ 成立，且 Sno 是 (Sno, Cno) 的真子集。



【定义】在 $R(U)$ 中，如果 $X \rightarrow Y$ ， $(Y \not\subseteq X)$ ， $Y \not\rightarrow X$ ， $Y \rightarrow Z$ ，则称 Z 对 X 传递函数依赖。

记为： $X \xrightarrow{\text{传递}} Z$

注：如果 $Y \rightarrow X$ ，即 $X \leftrightarrow Y$ ，则 Z 直接依赖于 X 。

例：在关系 $Std(Sno, Sdept, Mname)$ 中，有：

$Sno \rightarrow Sdept$ ， $Sdept \rightarrow Mname$

$Mname$ 传递函数依赖于 Sno



6.2.1 函数依赖

6.2.2 码

6.2.3 范式

6.2.4 2NF

6.2.5 3NF

6.2.6 BCNF

6.2.7 多值依赖

6.2.8 4NF

6.2.9 规范化小结



【定义】 设 K 为 $R(U, F)$ 中的属性或属性组合。若 $K \xrightarrow{F} U$ ，则 K 称为 R 的**候选码**(Candidate Key)。

若候选码多于一个，则选定其中的一个做为**主码**(Primary Key)。

■ 主属性与非主属性

- 包含在任何一个候选码中的属性，称为主属性 (Prime attribute)
- 不包含在任何码中的属性称为非主属性 (Nonprime attribute) 或非码属性 (Non-key attribute)

■ 全码

- 整个属性组是码，称为**全码** (All-key)



【例】 关系模式 $S(\underline{Sno}, Sdept, Sage)$ ，单个属性 Sno 是码，
 $SC(\underline{Sno}, \underline{Cno}, Grade)$ 中， (Sno, Cno) 是码

【例】 关系模式 $R(P, W, A)$

P ：演奏者 W ：作品 A ：听众

一个演奏者可以演奏多个作品

某一作品可被多个演奏者演奏

听众可以欣赏不同演奏者的不同作品

码为 (P, W, A) ，即All-Key



【定义】 关系模式 R 中属性或属性组 X 并非 R 的码，但 X 是另一个关系模式的码，则称 X 是 R 的**外部码(Foreign key)**也称外码。

【例】 $SC (\underline{Sno}, \underline{Cno}, Grade)$ 中， Sno 不是码，但 Sno 是关系模式 $S (\underline{Sno}, Sdept, Sage)$ 的码，则 Sno 是关系模式 SC 的外码

- 主码与外部码一起提供了表示关系间联系的手段。



6.2.1 函数依赖

6.2.2 码

6.2.3 范式

6.2.4 2NF

6.2.5 3NF

6.2.6 BCNF

6.2.7 多值依赖

6.2.8 4NF

6.2.9 规范化小结



- 范式是符合某一种级别的关系模式的集合
- 关系数据库中的关系必须满足一定的要求。满足不同程度要求的为不同范式

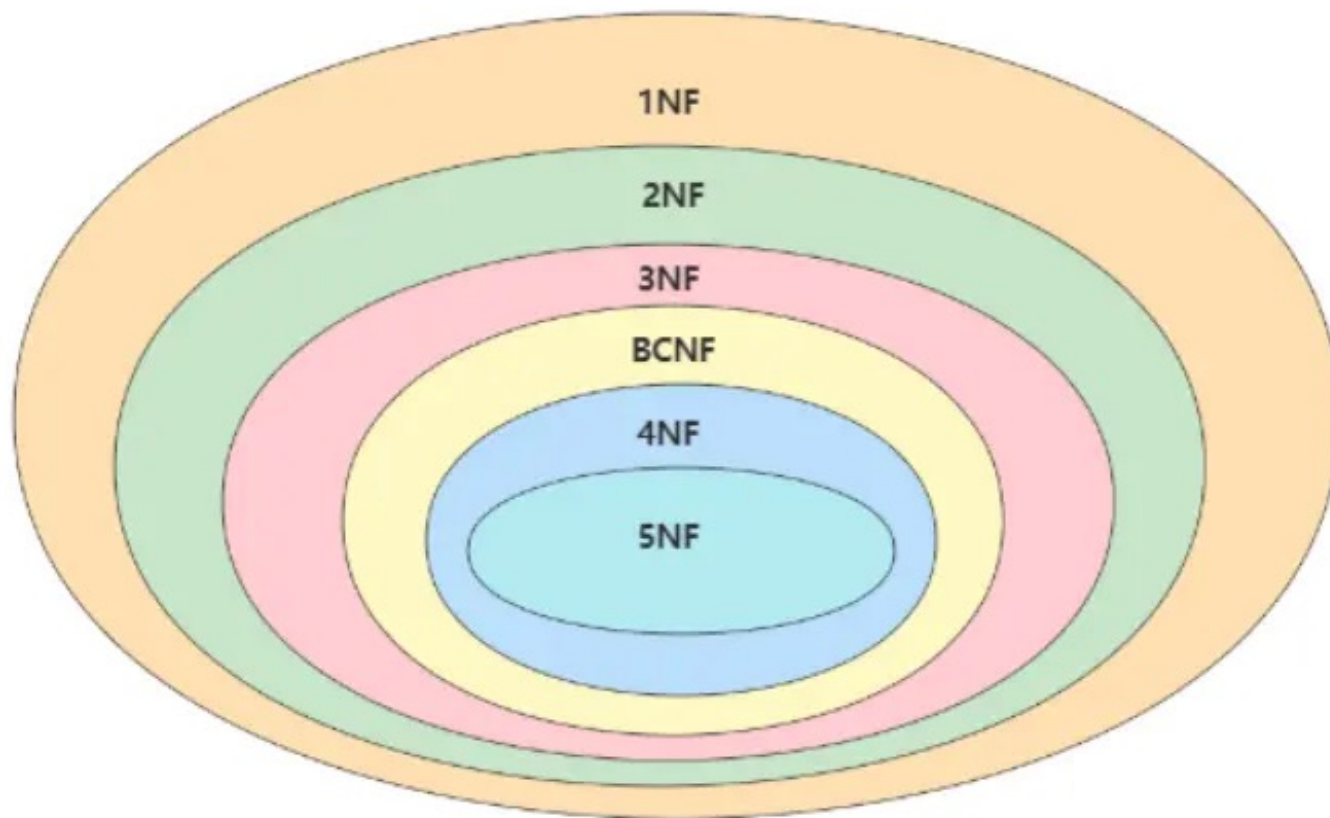
- 范式的种类：
 - 第一范式(1NF)
 - 第二范式(2NF)
 - 第三范式(3NF)
 - BC范式(BCNF)
 - 第四范式(4NF)
 - 第五范式(5NF)

$$1NF \supset 2NF \supset 3NF \supset BCNF \supset 4NF \supset 5NF$$

- 某一关系模式R为第n范式，可简记为 $R \in nNF$ 。
- 一个低一级范式的关系模式，通过模式分解可以转换为若干个高一级范式的关系模式的集合，这种过程就叫规范化



6种设计范式及其关系



$$1NF \supset 2NF \supset 3NF \supset BCNF \supset 4NF \supset 5NF$$



1NF的定义

如果一个关系模式R的所有属性都是**不可分的基本数据项**，则 $R \in 1NF$

- 第一范式是对关系模式的最起码的要求。不满足第一范式的数据库模式不能称为关系数据库
- 但满足第一范式的关系模式并不一定是一个好的关系模式

编号	品名	进货		销售		备注
		数量	单价	数量	单价	

编号	品名	进货数量	进货单价	销售数量	销售单价	备注



6.2.1 函数依赖

6.2.2 码

6.2.3 范式

6.2.4 2NF

6.2.5 3NF

6.2.6 BCNF

6.2.7 多值依赖

6.2.8 4NF

6.2.9 规范化小结



2NF的定义

若 $R \in 1NF$ ，且每一个**非主属性完全函数依赖于码**，则 $R \in 2NF$ 。

【例】关系模式 S-L-C(Sno, Sdept, Sloc, Cno, Grade)

Sloc为学生住处，假设每个系的学生住在同一个地方

函数依赖包括：

$(Sno, Cno) \xrightarrow{F} Grade$

$Sno \rightarrow Sdept$

$(Sno, Cno) \xrightarrow{P} Sdept$

$Sno \rightarrow Sloc$

$(Sno, Cno) \xrightarrow{P} Sloc$

$Sdept \rightarrow Sloc$



【例】关系模式 S-L-C(Sno, Sdept, Sloc, Cno, Grade)

Sloc为学生住处，假设每个系的学生住在同一个地方

函数依赖包括：

$(Sno, Cno) \xrightarrow{F} Grade$

$Sno \rightarrow Sdept$

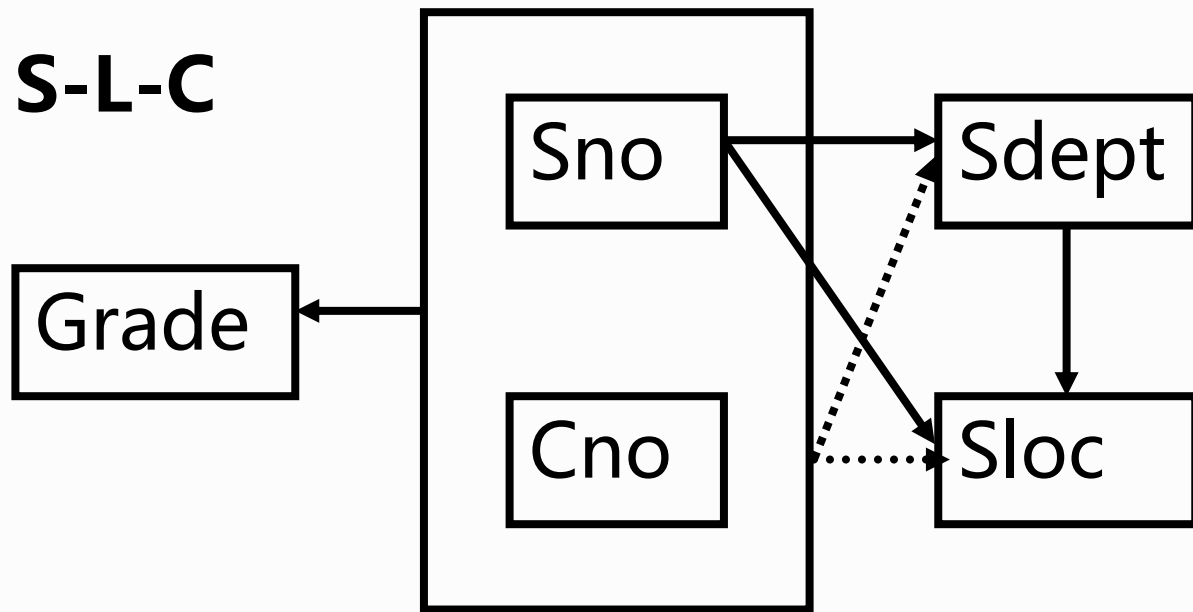
$(Sno, Cno) \xrightarrow{P} Sdept$

$Sno \rightarrow Sloc$

$(Sno, Cno) \xrightarrow{P} Sloc$

$Sdept \rightarrow Sloc$

S-L-C



- S-L-C的码为(Sno, Cno)
- S-L-C满足第一范式。
- 非主属性Sdept和Sloc部分函数依赖于码(Sno, Cno)



(1) 插入异常

若插入一个学生 $Sno=200201067, Sdept=Ma, Sloc=B01$, 但该生还没有选课, 即该学生无 Cno , 这样的元组无法插入 $S-L-C$ 。

(2) 删除异常

若假定某个学生只选择一门课程, 如 200201001 选择一门 $C3$ 。现学生不选 $C3$ 则将删除 $C3$ 数据项, 会造成学号信息也被删除。



(3) 修改复杂

- 若某个学生要求转系($MA \rightarrow CS$)，本来只需要修改此学生元组中的Sdept分量，但还必须修改学生住处分量Sloc。
- 如果该学生选修了K门课程，则Sdept、Sloc重复存储了K次，数据不仅冗余而且必须毫无遗漏地修改K个元组中的全部Sdept和Sloc属性值。



■ 原因

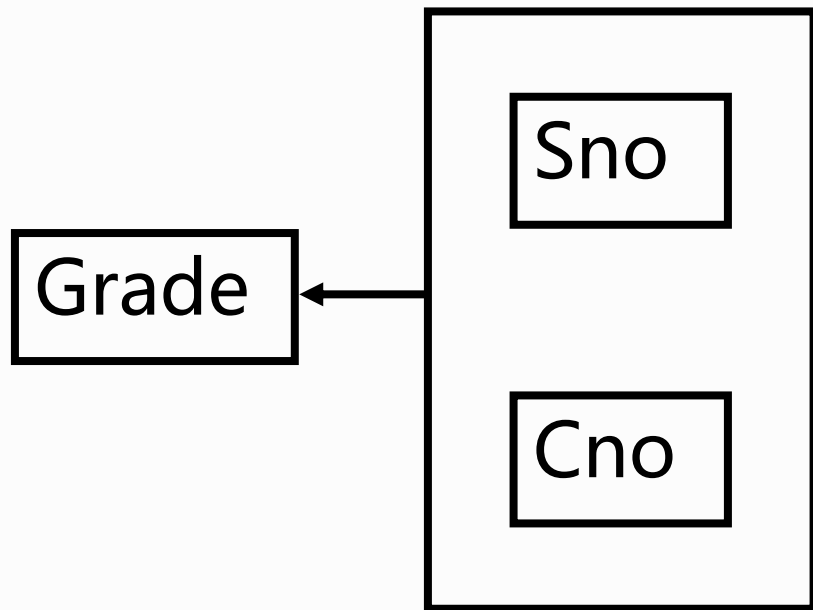
例子中有两类非主属性：一类如Grade，它对码完全函数依赖，另一类如Sdept、Sloc，它们对码不是完全函数依赖，而是部分函数依赖。

■ 解决方法

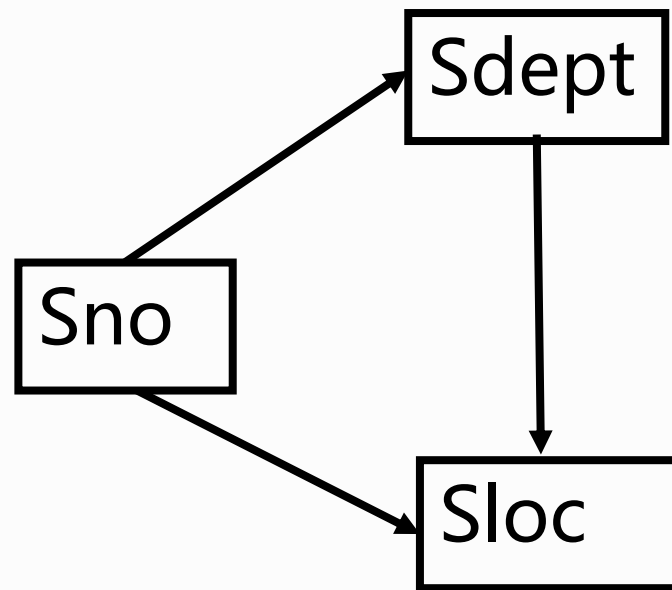
S-L-C分解为两个关系模式，以消除这些部分函数依赖

SC (Sno , Cno , Grade)

S-L (Sno , Sdept , Sloc)



SC中的函数依赖



S-L中的函数依赖

SC的码为(Sno, Cno)，SL的码为Sno，这样使得非主属性对码都是完全函数依赖了。



2NF的缺点：

➤ 数据冗余

每个系名和系主任的名字存储次数等于该系学生人数。

Student (Sno, Sdept, Mname, Cno, Grade) $\notin$ 2NF



SC (Sno, Cno, Grade) $\in$ 2NF

SD (Sno, Sdept, Mname) $\in$ 2NF



2NF的缺点：

$SC(Sno, Cno, Grade) \in 2NF$

$SD(Sno, Sdept, Mname) \in 2NF$

➤ 插入异常

当一个新系没有招生时，该系信息无法插入。

➤ 删除异常

如某系全部毕业而无招生时，删除全部学生信息也随之删除了该系信息。

➤ 更新异常

如更换系主任时，仍需改动较多学生的记录。



■ 2NF的缺点产生的原因：

➤ SD (Sno, Sdept, Mname) 中存在非主属性对主码的传递依赖。

$SD (Sno, Sdept, Mname) \in 2NF$

$Sno \rightarrow Sdept, Sdept \rightarrow Mname$

需要进一步对SD进行规范化



6.2.1 函数依赖

6.2.2 码

6.2.3 范式

6.2.4 2NF

6.2.5 3NF

6.2.6 BCNF

6.2.7 多值依赖

6.2.8 4NF

6.2.9 规范化小结



3NF的定义

【定义】关系模式 $R\langle U, F \rangle$ 中若不存在这样的码 X 、属性组 Y 及非主属性 Z ($Z \notin Y$), 使得 $X \rightarrow Y$, $Y \rightarrow Z$ 成立, $Y \nrightarrow X$, 则称 $R\langle U, F \rangle \in 3NF$ 。

➤ 若 $R \in 3NF$, 则每一个非主属性既不部分依赖于码也不传递依赖于码。



3NF定义

如果关系模式 $R \in 2NF$ ，且每个非主属性都不传递依赖于R的每个码，则 $R \in 3NF$ 。

- 属性不依赖于其它非主属性
- 不存在：关键字段 $\rightarrow$ 非关键字段x $\rightarrow$ 非关键字段y



【例】2NF关系模式S-L(Sno, Sdept, Sloc)中

— 函数依赖：

$Sno \rightarrow Sdept$

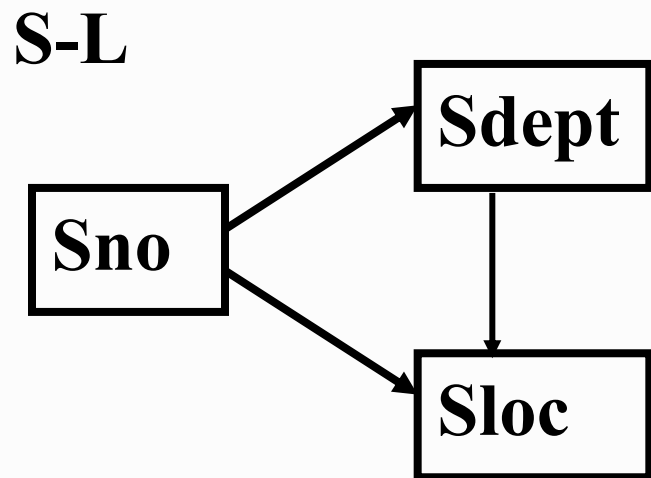
$Sdept \nrightarrow Sno$

$Sdept \rightarrow Sloc$

可得：

$Sno \xrightarrow{\text{传递}} Sloc$ ，即S-L中存在非主属性对码的传递函数依赖， $S-L \notin 3NF$

函数依赖图：





解决方法

采用投影分解法，把S-L分解为两个关系模式，以消除传递函数依赖：

S-D (Sno , Sdept)

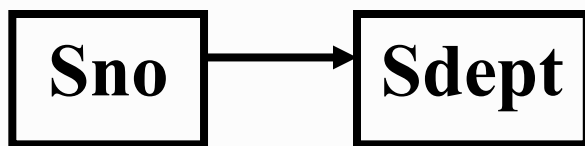
D-L (Sdept , Sloc)

S-D的码为Sno，D-L的码为Sdept。

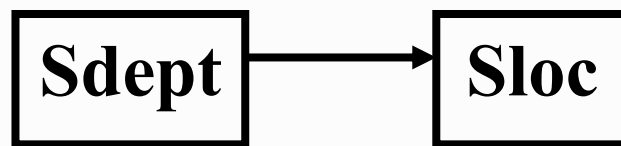
- **分解后的关系模式S-D与D-L中不再存在传递依赖**



S-D的码为Sno , D-L的码为Sdept



S-D



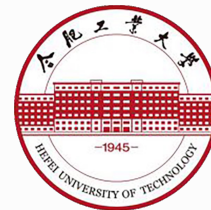
D-L

$S-L(Sno, Sdept, Sloc) \in 2NF$

$S-L(Sno, Sdept, Sloc) \notin 3NF$

$S-D(Sno, Sdept) \in 3NF$

$D-L(Sdept, Sloc) \in 3NF$



- 采用投影分解法将一个2NF的关系分解为多个3NF的关系，可以在一定程度上解决原2NF关系中存在的插入异常、删除异常、数据冗余度大、修改复杂等问题。
- **3NF要求每个非主键字段只依赖于主键，而不依赖于其他非主键字段**
- 只限制了非主属性对键的依赖关系，而没有限制主属性对键的依赖关系。如果发生这种依赖关系，仍有可能存在各种异常情况和数据冗余。



6.2.1 函数依赖

6.2.2 码

6.2.3 范式

6.2.4 2NF

6.2.5 3NF

6.2.6 BCNF

6.2.7 多值依赖

6.2.8 4NF

6.2.9 规范化小结



BCNF的定义（鲍依斯-科得范式）

【定义】 关系模式 $R\langle U, F \rangle \in 1NF$ ，若 $X \rightarrow Y$ 且 $Y \not\subseteq X$ 时 X 必含有码，则 $R\langle U, F \rangle \in BCNF$ 。

等价于：每一个决定属性因素都包含码。



- 若 $R \in BCNF$
 - 所有非主属性对每一个码都是完全函数依赖
 - 所有的主属性对每一个不包含它的码，也是完全函数依赖
 - 没有任何属性完全函数依赖于非码的任何一组属性



【例】关系模式C (Cno , Cname , Pcnno) , 它只有一个码**Cno** , 没有任何属性对**Cno**部分依赖或传递依赖 , 所以 **$C \in 3NF$** , 同时C中Cno是唯一的决定因素 , 所以 **$C \in BCNF$**

【例】关系模式S (Sno , Sname , Sdept , Sage) , 假定Sname具有唯一性 , 那么S就有两个码Sno , Sname , 这两个码都是由单个属性组成 , 彼此不相交。其他属性不存在对码的部分依赖或传递依赖 , 所以 **$S \in 3NF$** 。同时S中除Sno , Sname外**没有其他决定因素** , 所以 **$S \in BCNF$**



【例】关系模式SCP (S , C , P)

S-学生 C-课程 P-名次

语义：每个学生选修每门课程的成绩有一定的名次
每门课程的名次只有一个学生（即无并列）

- 函数依赖： $(S, C) \rightarrow P; (C, P) \rightarrow S$
- (S, C) 与 (C, P) 都可以作为候选码,属性相交
- 没有属性对码传递依赖或部分依赖，所以 $SCP \in 3NF$
- 除 (S, C) 和 (C, P) 以外没有其他决定因素，所以 $SCP \in BCNF$



【例】在关系模式STC (S , T , C) 中。

S表示学生，T表示教师，C表示课程。

语义：每个教师只教一门课程；

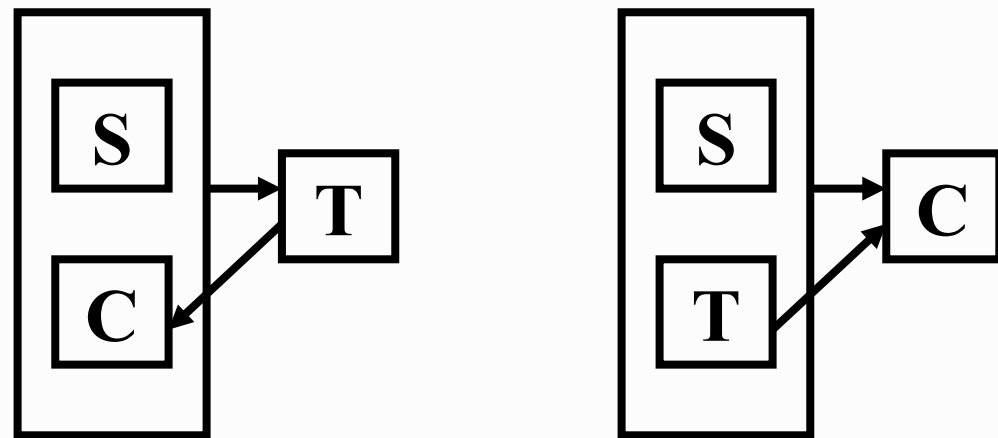
每门课程有若干教师；

某一学生选定某门课程就对应一个固定教师。

— 函数依赖：

$(S, C) \rightarrow T, (S, T) \rightarrow C, T \rightarrow C$

— (S, C)和(S, T)都是候选码



STC中的函数依赖



【例】在关系模式STC (S , T , C) 中。

S表示学生，T表示教师，C表示课程。

语义：每个教师只教一门课程；

每门课程有若干教师；

某一学生选定某门课程就对应一个固定教师。

– 函数依赖：

$(S, C) \rightarrow T, (S, T) \rightarrow C, T \rightarrow C$

– (S, C)和(S, T)都是候选码

• $STC \in 3NF$

– 没有任何非主属性对码传递依赖或部分依赖

• $STC \notin BCNF$

– T是决定因素，T不包含码



STC存在的问题：

➤ 数据冗余

虽然每个教师只开一门课程，但是每个选修该教师该门课程的学生元组都要记录课程信息。

➤ 插入异常

某门课程本学期不开设，自然就没有学生选修，造成教师上该门课程的信息无法插入；学生刚入校尚未选课，有关信息也无法输入。

➤ 删除异常

若选修某门课程的学生全部毕业，删除学生信息的同时随之也删除了教师开设该门课程的信息。

➤ 更新异常

当教师开设的某门课程修更名后，所有选修该教师的该门课程的学生元组都要进行修改，若漏改数据会破坏数据的完整性。

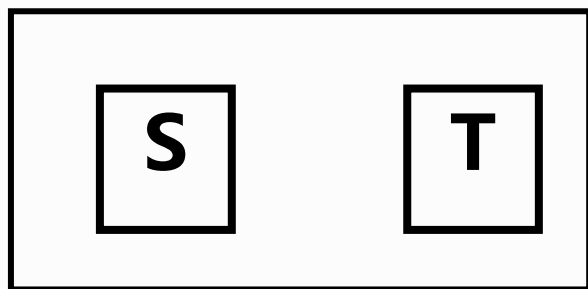


STC存在问题的原因：

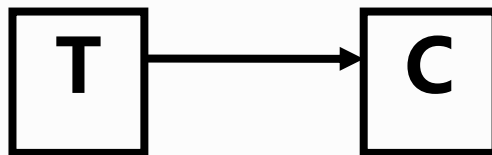
主属性部分依赖于码： $(S,T) \xrightarrow{P} C$

■ 解决方法：将STC分解为二个关系模式：

$ST(S, T) \in BCNF$, $TC(T, C) \in BCNF$



ST



TC

没有任何属性对码的部分函数依赖和传递函数依赖

在第三范式基础上，要求所有**非主码属性都必须依赖于主码。**



STC分解后，消除了存在的异常问题：

- 数据冗余降低

每个教师开设课程的信息只在TC中存储一次。

- 不存在插入异常

所开设课程尚无学生选修的教师信息可以直接存储在关系TC中；而尚无选修课程的学生可以存储在关系ST中。

- 不存在删除异常

若选修某门课程的学生全部毕业，可以只删除关系ST中的相关学生信息，而不影响TC中相应教师开设课程的信息。

- 更新异常

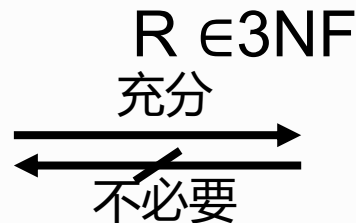
当某个教师开设的某门课程修更名后，只需修改关系TC中的一个相应元组即可，不会破坏数据完整性。



■ 3NF很大程度上消除了插入异常和删除异常，但可能存在主属性对候选键的部分依赖和传递依赖，因此关系模式的分解仍不够。

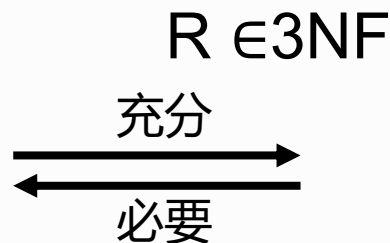
■ BCNF已经实现了模式的彻底分解，消除了插入异常和删除异常，而且数据的冗余也减少到了极小程度。

- $R \in \text{BCNF}$



- 如果 $R \in 3\text{NF}$ ，且 R 只有一个候选码

$R \in \text{BCNF}$





6.2.1 函数依赖

6.2.2 码

6.2.3 范式

6.2.4 2NF

6.2.5 3NF

6.2.6 BCNF

6.2.7 多值依赖

6.2.8 4NF

6.2.9 规范化小结



【例】 学校中某一门课程由多个教师讲授，他们使用相同的一套参考书。每个教员可以讲授多门课程，每种参考书可以供多门课程使用。

■ 非规范化关系

课 程 C	教 员 T	参 考 书 B
物理	$\left\{ \begin{array}{l} \text{李 勇} \\ \text{王 军} \end{array} \right\}$	$\left\{ \begin{array}{l} \text{普通物理学} \\ \text{光学原理} \\ \text{物理习题集} \end{array} \right\}$
数学	$\left\{ \begin{array}{l} \text{李 勇} \\ \text{张 平} \end{array} \right\}$	$\left\{ \begin{array}{l} \text{数学分析} \\ \text{微分方程} \\ \text{高等代数} \end{array} \right\}$
计算数学	$\left\{ \begin{array}{l} \text{张 平} \\ \text{周 峰} \end{array} \right\}$	$\left\{ \begin{array}{l} \text{数学分析} \\ \dots \\ \dots \end{array} \right\}$
⋮	⋮	⋮



■ 用二维表表示Teaching

- Teaching ∈ BCNF
- Teaching 具有唯一候选码(C, T, B), 即全码

Teaching模式中存在的问题

- (1)数据冗余度大：有多少名任课教师，参考书就要存储多少次
- (2)插入操作复杂：当某一课程增加一名任课教师时，课程有多少本参考书，就必须插入多少个元组
- (3)删除操作复杂：某一门课要去掉一本参考书，该课有多少名教师，就必须删除多少个元组
- (4)修改操作复杂：某一门课要修改一本参考书，该课程有多少名教师，就必须修改多少个元组

课程C	教员T	参考书B
物 理	李 勇	普通物理学
物 理	李 勇	光学原理
物 理	李 勇	物理习题集
物 理	王 军	普通物理学
物 理	王 军	光学原理
物 理	王 军	物理习题集
数 学	李 勇	数学分析
数 学	李 勇	微分方程
数 学	李 勇	高等代数
数 学	张 平	数学分析
数 学	张 平	微分方程
数 学	张 平	高等代数
...	...	...



【定义】设 $R(U)$ 是一个属性集 U 上的一个关系模式， X 、 Y 和 Z 是 U 的子集，并且 $Z = U - X - Y$ 。关系模式 $R(U)$ 中**多值依赖** $X \twoheadrightarrow Y$ 成立，当且仅当对 $R(U)$ 的任一关系 r ，给定的一对 (x, z) 值，有**一组** Y 的值，这组值仅仅决定于 x 值而与 z 值无关（**一个 x 对应多个 y** ）

【例】在关系 $\text{Teaching}(C, T, B)$ 中，对于 C 的每一个值， T 有一组值与之对应，而不论 B 取何值。因此 T 多值依赖于 C ，即 $C \twoheadrightarrow T$ 。（ **C 一个值能确定 T 一个集合**）

课 程 C	教 员 T	参 考 书 B
-------	-------	---------



多值依赖的另一个等价的形式化的定义：

在 $R(U)$ 的任一关系 r 中，如果存在元组 t, s 使得 $t[X] = s[X]$ ，那么就必然存在元组 $w, v \in r$ ，（ w, v 可以与 s, t 相同），使得 $w[X] = v[X] = t[X]$ ，而 $w[Y] = t[Y]$ ， $w[Z] = s[Z]$ ， $v[Y] = s[Y]$ ， $v[Z] = t[Z]$ （即交换 s, t 元组的 Y 值所得的两个新元组必在 r 中），则 Y 多值依赖于 X ，记为 $X \twoheadrightarrow Y$ 。这里， X, Y 是 U 的子集， $Z = U - X - Y$ 。

■ 平凡多值依赖和非平凡的多值依赖

- 若 $X \twoheadrightarrow Y$ ，而 $Z = \emptyset$ ，则称 $X \twoheadrightarrow Y$ 为**平凡的多值依赖**
- 否则称 $X \twoheadrightarrow Y$ 为**非平凡的多值依赖**

t

s

课程C	教员T	参考书B
物 理	李 勇	普通物理学
物 理	李 勇	光学原理
物 理	李 勇	物理习题集
物 理	王 军	普通物理学
物 理	王 军	光学原理
物 理	王 军	物理习题集
数 学	李 勇	数学分析
数 学	李 勇	微分方程
数 学	李 勇	高等代数
数 学	张 平	数学分析
数 学	张 平	微分方程
数 学	张 平	高等代数
...	...	...



【例】关系模式WSC (W , S , C)

- W表示仓库，S表示保管员，C表示商品
- 假设每个仓库有若干个保管员，有若干种商品
- 每个保管员保管所在的仓库的所有商品
- 每种商品被所有保管员保管

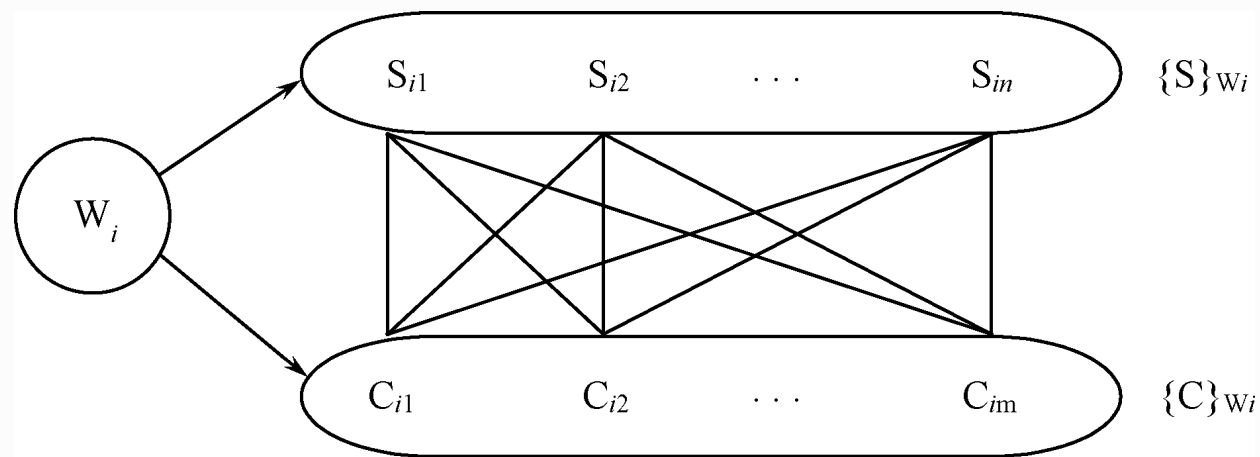
对于W的每一个值 W_i ，S有一个完整的集合
与之对应且不问C取何值，所以 $W \rightarrow S$
对于W的每一个值 W_i ，C有一个完整的集合
与之对应且不问S取何值，所以 $W \rightarrow C$

W	S	C
W1	S1	C1
W1	S1	C2
W1	S1	C3
W1	S2	C1
W1	S2	C2
W1	S2	C3
W2	S3	C4
W2	S3	C5
W2	S4	C4
W2	S4	C5



W	S	C
W1	S1	C1
W1	S1	C2
W1	S1	C3
W1	S2	C1
W1	S2	C2
W1	S2	C3
W2	S3	C4
W2	S3	C5
W2	S4	C4
W2	S4	C5

用下图表示这种对应



$W \twoheadrightarrow S$ 且 $W \twoheadrightarrow C$

一个属性的值可以确定一组属性



【说明】

- 对应 W 的某一个值 W_i 的全部 S 值记作 $\{S\}_{W_i}$ （表示此仓库工作的全部保管员）
- 全部 C 值记作 $\{C\}_{W_i}$ （表示在此仓库中存放的所有商品）
- 应当有 $\{S\}_{W_i}$ 中的每一个值和 $\{C\}_{W_i}$ 中的每一个 C 值对应
- 于是 $\{S\}_{W_i}$ 与 $\{C\}_{W_i}$ 之间正好形成一个完全二分图，故 $W \rightarrow \rightarrow S$
- 由于 C 与 S 的完全对称性，必然有 $W \rightarrow \rightarrow C$ 成立



多值依赖的性质

(1) 多值依赖具有对称性

若 $X \twoheadrightarrow Y$ ，则 $X \twoheadrightarrow Z$ ，其中 $Z = U - X - Y$

每个保管员保管所有的商品，每个商品被所有保管员保管，显然若 $W \twoheadrightarrow S$ ，则必然有 $W \twoheadrightarrow C$

(2) 多值依赖具有传递性

若 $X \twoheadrightarrow Y$ ， $Y \twoheadrightarrow Z$ ，则 $X \twoheadrightarrow Z - Y$

(3) 函数依赖是多值依赖的特殊情况。

若 $X \rightarrow Y$ ，则 $X \twoheadrightarrow Y$ 。

(4) 若 $X \twoheadrightarrow Y$ ， $X \twoheadrightarrow Z$ ，则 $X \twoheadrightarrow Y \cup Z$ 。

(5) 若 $X \twoheadrightarrow Y$ ， $X \twoheadrightarrow Z$ ，则 $X \twoheadrightarrow Y \cap Z$ 。

(6) 若 $X \twoheadrightarrow Y$ ， $X \twoheadrightarrow Z$ ，则 $X \twoheadrightarrow Y - Z$ ， $X \twoheadrightarrow Z - Y$ 。



多值依赖与函数依赖的区别

(1) 多值依赖的有效性与属性集的范围有关

若 $X \twoheadrightarrow Y$ 在 U 上成立，则在 $W (XY \subseteq W \subseteq U)$ 上一定成立；反之则不然，即 $X \twoheadrightarrow Y$ 在 $W (W \subseteq U)$ 上成立，在 U 上**不一定成立**。

原因：多值依赖的定义中**不仅涉及属性组 X 和 Y** ，而且涉及 U 中**其余属性 Z** 。

一般地，在 $R(U)$ 上若有 $X \twoheadrightarrow Y$ 在 $W (W \subseteq U)$ 上成立，则称 $X \twoheadrightarrow Y$ 为 $R(U)$ 的**嵌入型多值依赖**。

函数依赖 $X \rightarrow Y$ 的有效性仅决定于 X 、 Y 这两个属性集的值

只要在 $R(U)$ 的任何一个关系 r 中，元组在 X 和 Y 上的值满足函数依赖定义，则函数依赖 $X \rightarrow Y$ 在任何属性集 $W (XY \subseteq W \subseteq U)$ 上成立。



多值依赖与函数依赖的区别

(2) 若函数依赖 $X \rightarrow Y$ 在 $R(U)$ 上成立, 则对于任何 $Y' \subseteq Y$ 均有 $X \rightarrow Y'$ 成立

多值依赖 $X \twoheadrightarrow Y$ 若在 $R(U)$ 上成立, 不能断言对于任何 $Y' \subseteq Y$ 均有 $X \twoheadrightarrow Y'$ 成立

【例】关系 $R(A, B, C, D)$, $A \twoheadrightarrow BC$ 成立, 也有 $A \twoheadrightarrow D$ 。在此实例上 $A \twoheadrightarrow B$ 是不成立的

A	B	C	D
a_1	b_1	c_1	d_1
a_1	b_1	c_1	d_2
a_1	b_2	c_2	d_1
a_1	b_2	c_2	d_2



6.2.1 函数依赖

6.2.2 码

6.2.3 范式

6.2.4 2NF

6.2.5 3NF

6.2.6 BCNF

6.2.7 多值依赖

6.2.8 4NF

6.2.9 规范化小结



【定义】关系模式 $R(U, F) \in 1NF$ ，如果对于 R 的每个非平凡多值依赖 $X \twoheadrightarrow Y$ ($Y \not\subseteq X$)， X 都**含有**码，则 $R \in 4NF$ 。

- 4NF是限制关系模式的属性之间不允许有非平凡且非函数依赖的多值依赖。
4NF所允许的非平凡多值依赖实际是函数依赖
- 如果 $R \in 4NF$ ，则 $R \in BCNF$
 - **不允许**有非平凡且非函数依赖的**多值依赖**
 - **允许**的非平凡多值依赖是**函数依赖** ($X \rightarrow Y$)
- 在关系WSC中， $W \twoheadrightarrow S$ ， $W \twoheadrightarrow C$ ，他们都是非平凡多值依赖。而W不是码，关系模式的码是(W, S, C)，即全码，因此WSC $\notin 4NF$ 。可以把WSC分解成WS(W, S)，WC(W, C)， $WS \in 4NF$ ， $WC \in 4NF$



【例】Teaching(C,T,B) $\notin$ 4NF

存在非平凡的多值依赖 $C \twoheadrightarrow T$ ，且C不是码

- 用投影分解法把Teaching分解为如下两个关系模式：

$CT(C, T) \in 4NF$

$CB(C, B) \in 4NF$

$C \twoheadrightarrow T$ ， $C \twoheadrightarrow B$ 是平凡多值依赖

课 程 C	教 员 T	参 考 书 B
-------	-------	---------



6.2.1 函数依赖

6.2.2 码

6.2.3 范式

6.2.4 2NF

6.2.5 3NF

6.2.6 BCNF

6.2.7 多值依赖

6.2.8 4NF

6.2.9 规范化小结



小结

- 在关系数据库中，对关系模式的基本要求是满足第一范式。
- 规范化程度过低的关系不一定能够很好地描述现实世界。可能存在插入异常、删除异常、修改复杂，数据冗余等问题，解决方法就是对其进行规范化，转换成高级范式。
- 一个低一级范式的关系模式，通过模式分解可以转换为若干个高一级范式的关系模式集合，这种过程就叫关系模式的规范化。
- 关系数据库的规范化理论是数据库逻辑设计的工具。



规范化的基本思想

- 逐步消除数据依赖中不合适的部分，使模式中的各关系模式达到某种程度的“分离”

- 采用“一事一地”的模式设计原则

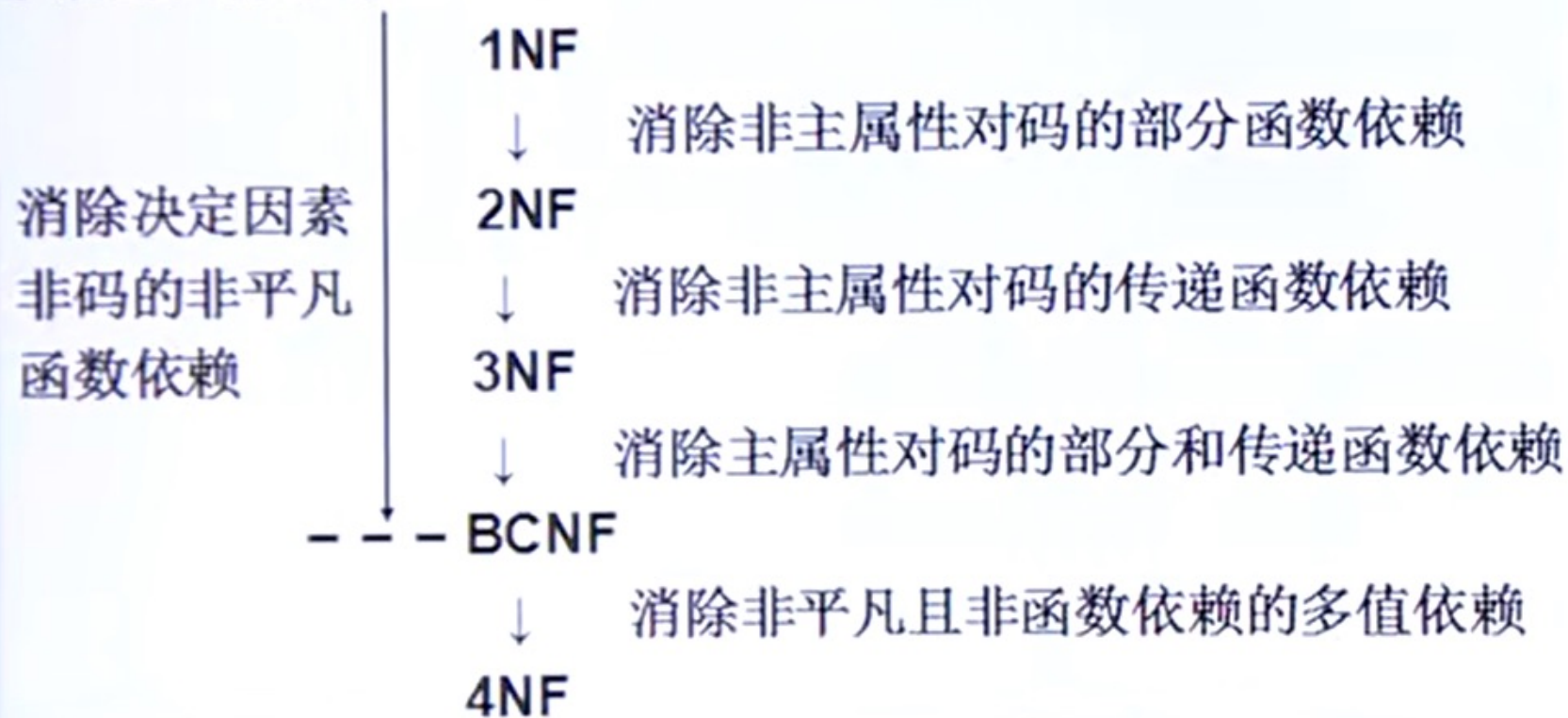
让一个关系描述一个概念、一个实体或者实体间的一种联系。若多于一个概念就把它“分离”出去，因此规范化实质上是概念的单一化。

- **不能说规范化程度越高的关系模式就越好。**必须对现实世界的实际情况和用户应用需求作进一步分析，确定一个合适的、能够反映现实世界的模式。



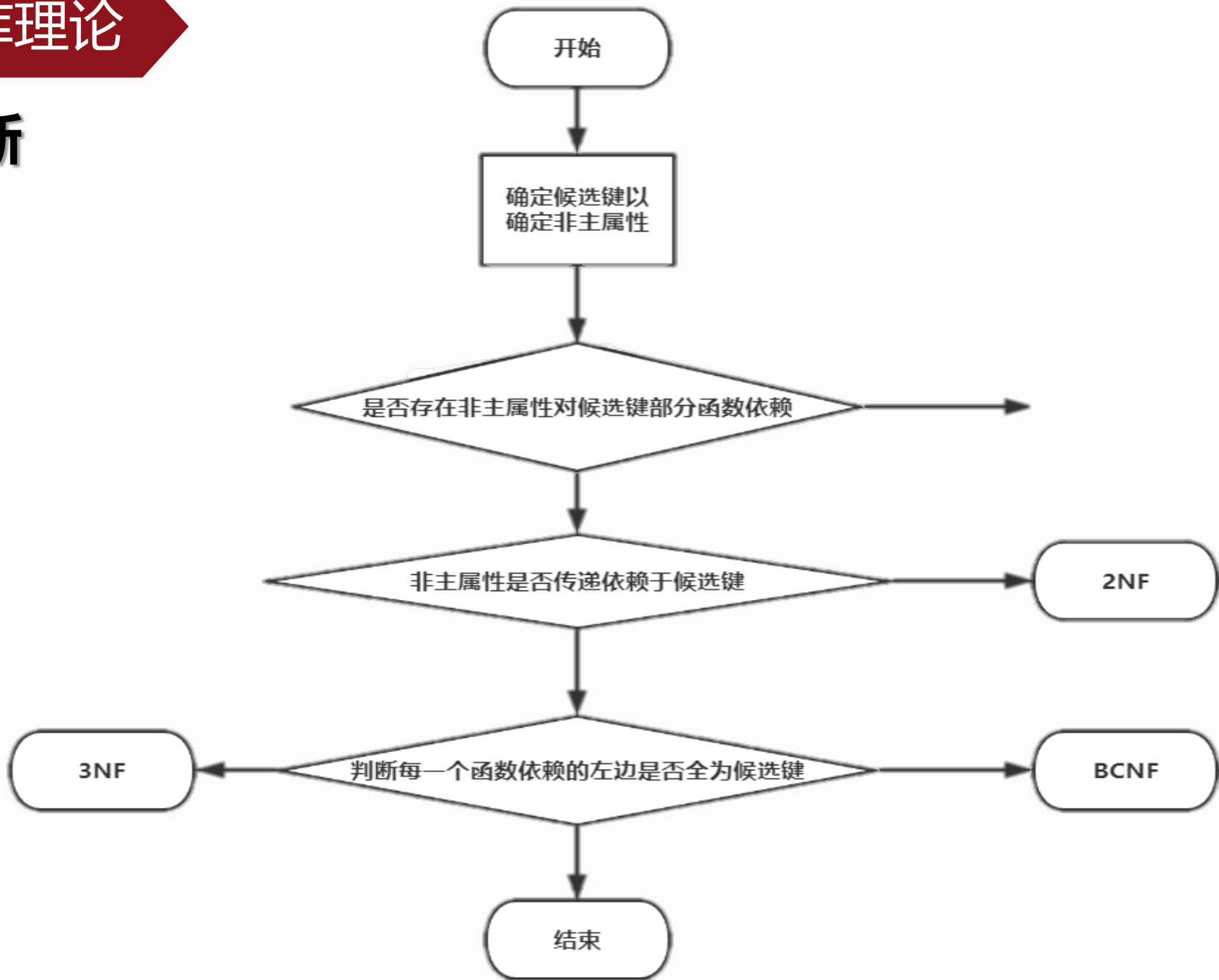
规范化过程

关系模式规范化的基本步骤





关系范式判断





【练习1】 $R(X,Y,Z) F=(XY \rightarrow Z)$

(1) 确定候选码和非主属性：

R的候选码为XY，非主属性是Z

**(2) 是否存在非主属性对候选码部分函数依赖
不存在**

**(3) 是否存在非主属性传递依赖于候选码
不存在**

**(4) 函数依赖左边是否全为候选码
函数依赖的左边XY为候选码**

因此该范式达到BCNF



【练习2】 $R(X,Y,Z)$ $F=(Y \rightarrow Z, XZ \rightarrow Y)$

(1) 确定候选码和非主属性：

R的候选码为XY和XZ，非主属性：无

因为非主属性的不存在所以至少达到3NF（即第2、3步的非主属性部分依赖和传递依赖不用判断了）

(2) 函数依赖左边是否全为候选码

第一个函数依赖 $Y \rightarrow Z$ 的左边Y不是候选键

因此此范式达到3NF



【练习3】 $R(X,Y,Z)$ $F=(Y \rightarrow Z, Y \rightarrow X, X \rightarrow YZ)$

(1) 确定候选码和非主属性：

R的候选码为X和Y.非主属性:Z

(2) 是否存在非主属性对候选码部分函数依赖：

不存在

(3) 是否存在非主属性传递依赖于候选码：

不存在

(4) 函数依赖的左边都是候选码

因此此范式达到BCNF



【练习4】 $R(A,B,C,D)$ $F=\{B \rightarrow C, C \rightarrow D, D \rightarrow A\}$

(1) 确定候选码和非主属性：

R的候选码为B，非主属性：A,C,D

(2) 是否存在非主属性对候选码部分函数依赖：

候选码为单属性，因此不存在

(3) 是否存在非主属性传递依赖于候选码：

由 $B \rightarrow C, C \rightarrow D$ 且C不能推出B可知此范式不满足3NF

(4) 函数依赖的左边都是候选码

因此此范式达到2NF



【练习5】 $R(S,T,J)$ $F=\{SJ \rightarrow T, ST \rightarrow J, T \rightarrow J\}$

(1) 确定候选码和非主属性：

R的候选码为SJ和ST，非主属性：无

因为非主属性的不存在所以至少达到3NF

(2) 函数依赖的左边都是候选码

由 $T \rightarrow J$ 左边T不是候选码

因此此范式达到3NF



关系数据库理论



6.1 问题的提出

6.2 规范化

6.3 数据依赖的公理系统

6.4 模式的分解



逻辑蕴涵

【定义】对于满足一组**函数依赖** F 的关系模式 $R(U, F)$ ，其任何一个关系 r ，若函数依赖 $X \rightarrow Y$ 都成立，（即 r 中任意两元组 t, s ，若 $t[X] = s[X]$ ，则 $t[Y] = s[Y]$ ），则称 F **逻辑蕴涵** $X \rightarrow Y$



Armstrong公理系统

- 一套**推理规则**，是模式分解算法的理论基础。
- 用途：
 - （1）求函数依赖集闭包、属性集闭包。
 - （2）求最小函数依赖集。
 - （3）求给定关系模式的候选码。
 - （4）模式分解。



关系模式 $R(U, F)$ 来说有以下的推理规则：

- A1. **自反律** (Reflexivity) : 若 $Y \subseteq X \subseteq U$, 则 $X \rightarrow Y$ 为 F 所蕴含。
- A2. **增广律** (Augmentation) : 若 $X \rightarrow Y$ 为 F 所蕴含, 且 $Z \subseteq U$, 则 $XZ \rightarrow YZ$ 为 F 所蕴含。
- A3. **传递律** (Transitivity) : 若 $X \rightarrow Y$ 及 $Y \rightarrow Z$ 为 F 所蕴含, 则 $X \rightarrow Z$ 为 F 所蕴含。

注意：自反律所得到的函数依赖均是平凡的函数依赖，自反律的使用并不依赖于 F



证明Armstrong推理规则的正确性

(I) A1自反律: 若 $Y \subseteq X \subseteq U$, 则 $X \rightarrow Y$ 为 F 所蕴含

证: 设 $Y \subseteq X \subseteq U$

- $R(U, F)$ 的 $\forall r$ 中, 对 $\forall$ 元组 t, s :
- 若 $t[X] = s[X]$, (1)
- $\because Y \subseteq X$, 则 $t[Y] = s[Y]$, (2)
- $\therefore$ 由(1),(2)可得 $X \rightarrow Y$ 成立, 自反律得证。

如: $(sno, sname) \rightarrow sname$



证明Armstrong推理规则的正确性

(2) A2增广律: 若 $X \rightarrow Y$ 为 F 所蕴含, 且 $Z \subseteq U$, 则 $XZ \rightarrow YZ$ 为 F 所蕴含。

证: 设 $X \rightarrow Y$ 为 F 所蕴含, 且 $Z \subseteq U$ 。

设 $R(U, F)$ 的任一关系 r 中任意的两个元组 t, s :

若 $t[XZ] = s[XZ]$, 则有 $t[X] = s[X]$ 和 $t[Z] = s[Z]$;

由 $X \rightarrow Y$, 于是有 $t[Y] = s[Y]$, 所以 $t[YZ] = s[YZ]$, 所以 $XZ \rightarrow YZ$ 为 F 所蕴含, 增广律得证。

如: $\text{sno} \rightarrow \text{sname}$, $\text{sdept} \subseteq U$, 则 $(\text{sno}, \text{sdept}) \rightarrow (\text{sname}, \text{sdept})$



证明Armstrong推理规则的正确性

(3) A3传递律：若 $X \rightarrow Y$ 及 $Y \rightarrow Z$ 为 F 所蕴含，则 $X \rightarrow Z$ 为 F 所蕴含。

证：设 $X \rightarrow Y$ 及 $Y \rightarrow Z$ 为 F 所蕴含。

对 $R(U, F)$ 的任一关系 r 中的任意两个元组 t, s ：

若 $t[X] = s[X]$ ，由于 $X \rightarrow Y$ ，有 $t[Y] = s[Y]$ ；

再由 $Y \rightarrow Z$ ，有 $t[Z] = s[Z]$ ，所以 $X \rightarrow Z$ 为 F 所蕴含，传递律得证。

如：sno→sdept, sdept→dname
可得：sno→dname



导出规则

1. 根据A1, A2, A3这三条推理规则可以得到下面三条推理规则：

– **合并规则**：由 $X \rightarrow Y, X \rightarrow Z$, 有 $X \rightarrow YZ$ 。

(A2 , A3)

若已知 $sno \rightarrow sname, sno \rightarrow sdept$
可得： $sno \rightarrow sname, sdept$

– **伪传递规则**：由 $X \rightarrow Y, WY \rightarrow Z$, 有 $XW \rightarrow Z$ 。

(A2 , A3)

– **分解规则**：由 $X \rightarrow Y$ 及 $Z \subseteq Y$, 有 $X \rightarrow Z$ 。

(A1 , A3)

若已知： $sno \rightarrow sname, sdept$
可得： $sno \rightarrow sname, sno \rightarrow sdept$



【例】设有关系模式R，A、B、C、D、E、F是它的属性子集，R满足下列函数依赖：

$$F = \{A \rightarrow BC, CD \rightarrow EF\}$$

证明：函数依赖 $AD \rightarrow F$ 成立。

证明：	$A \rightarrow BC$	条件
	$A \rightarrow C$	分解规则
	$AD \rightarrow CD$	增广律
	$CD \rightarrow EF$	条件
	$AD \rightarrow EF$	传递律
	$AD \rightarrow F$	分解规则



导出规则

2.根据合并规则和分解规则，可得引理6.1

引理6.1 $X \rightarrow A_1 A_2 \dots A_k$ 成立的充分必要条件是 $X \rightarrow A_i$ 成立 ($i=1, 2, \dots, k$)



函数依赖闭包

【定义】由被 F 逻辑蕴涵的函数依赖的全体构成的集合，称为 F 的闭包，记作 F^+

自反律、传递律和增广律称为Armstrong公理系统

Armstrong公理系统具有有效性和完备性

- **有效性**：由 F 出发根据Armstrong公理推导出来的每一个函数依赖一定在 F^+ 中；
- **完备性**： F^+ 中的每一个函数依赖，必定可以由 F 出发根据Armstrong公理推导出来



函数依赖闭包

【定义】 设 F 为属性集 U 上的一组函数依赖， $X \subseteq U$ ， $X_F^+ = \{ A | X \rightarrow A \text{ 能由 } F \text{ 根据 Armstrong 公理导出} \}$ ， X_F^+ 称为属性集 X 关于函数依赖集 F 的闭包。

【引理】 设 F 为属性集 U 上的一组函数依赖， $X, Y \subseteq U$ ， $X \rightarrow Y$ 能由 F 根据 Armstrong 公理导出的充分必要条件是 $Y \subseteq X_F^+$

【用途】 将判定 $X \rightarrow Y$ 是否能由 F 根据 Armstrong 公理导出的问题，转化为求出 X_F^+ 、判定 Y 是否为 X_F^+ 的子集的问题



【算法】求属性集 $X (X \subseteq U)$ 关于 U 上的函数依赖 F 的闭包 X_F^+

输入： X, F 输出 X_F^+

步骤

即在 F 中寻找其左边为 $X(i)$ 的子集的函数依赖，将其右边在 $X(i)$ 中未出现过的属性组成的新集合 B

(1) $X^{(i)} = X, i=0;$

(2) 求 B ，这里 $B = \{A / (\exists V)(\exists W)((V \rightarrow W \in F) \wedge (V \subseteq X) \wedge (A^{(i)} \in W))\}$

(3) $X^{(i+1)} = B \cup X^{(i)}$

(4) 判断 $X^{(i+1)} = X^{(i)}$

(5) 若 $X^{(i+1)}$ 与 $X^{(i)}$ 相等或 $X^{(i)} = U$ ，则 $X^{(i)}$ 就是 X_F^+ ，算法终止。

(6) 否则，则 $i=i+1$ ，返回第(2)步



【例】已知关系模式 $R\langle U, F\rangle$, 其中

$U=\{A, B, C, D, E\}$;

$F=\{AB\rightarrow C, B\rightarrow D, C\rightarrow E, EC\rightarrow B, AC\rightarrow B\}$ 。

求 $(AB)_F^+$ 。

解 设 $X^{(0)}=AB$;

(1) 计算 $X^{(1)}$: 逐一扫描 F 集合中各个函数依赖, 找出左边是 A 、 B 或 AB 的函数依赖, 得到两个 $AB\rightarrow C, B\rightarrow D$, 所以 $X^{(1)}=AB\cup CD=ABCD$ 。

(2) 因为 $X^{(0)}\neq X^{(1)}$, 所以再找出左边是 $ABCD$ 子集的函数依赖, 得到 $C\rightarrow E, AC\rightarrow B$, 于是 $X^{(2)}=X^{(1)}\cup BE=ABCDE$ 。

(3) $X^{(2)}=U$, 算法终止

$\rightarrow (AB)_F^+=ABCDE$ 。



【例】 $U = \{A, B, C, D\}$; $F = \{A \rightarrow B, BC \rightarrow D, B \rightarrow C\}$;

(1) 求 A_F^+

解： $X(0) = A$

$\therefore X(1) = X(0) \cup B = AB$, 显然 $X(1) \neq X(0)$ 。

$\therefore X(2) = X(1) \cup C = ABC$, 显然 $X(1) \neq X(2)$ 。

$\therefore X(3) = X(2) \cup D = ABCD = U$, 循环终止。

$\therefore A_F^+ = ABCD$ 。

则再在F中找出左边是ABC子集的函数依赖，其结果是：
 $A \rightarrow B$, $B \rightarrow C$, $BC \rightarrow D$ 。则D将加入闭包集中。



【例】 $U=\{A, B, C, D\}$; $F=\{A\rightarrow B, BC\rightarrow D, B\rightarrow C\}$;

(2) 求 B_F^+

解：

$X(0)=B$

$X(1)=BCD$

$X(2)=BCD=X(1)$, 循环终止。所以 $B_F^+ = BCD$ 。



【例】 $U=\{A, B, C, D\}$; $F=\{A\rightarrow B, BC\rightarrow D, B\rightarrow C\}$;

(3) 求 C_F^+

解：

$X(0)=C$

$X(1)=C=X(0)$, 循环终止。所以 $C_F^+ = C$ 。



【练习】 $U=\{A, B, C, D, E\}$; $F=\{AB\rightarrow C, B\rightarrow D, C\rightarrow E, EC\rightarrow B, AC\rightarrow B\}$;

求 C_F^+

解：

(1) $X^{(0)}=C$

(2) $X^{(1)}$ ，在 F 中找左边为 C 的函数依赖，得到 $C\rightarrow E$ 。于是 $X(1) = C \cup E = CE$ 。

(3) 因为 $X^{(1)} \neq X^{(0)}$ ，所以再找出左部为 CE 子集的那些依赖函数，得到 $EC \rightarrow B$ ，于是 $X^{(2)} = X^{(1)} \cup B = BCE$

(4) 因为 $X^{(2)} \neq X^{(1)}$ ，所以再找出左部为 BCE 子集的那些依赖函数，得到 $B\rightarrow D$ ，于是 $X^{(3)} = X^{(2)} \cup D = BCDE$

(5) 因为 $X^{(3)} \neq X^{(2)}$ ，所以再找出左部为 $BCDE$ 子集的那些依赖函数，得到 $\emptyset$ ，于是 $X^{(4)} = X^{(3)} = BCDE$ ，循环终止。



通过函数依赖集F确定候选码（思考候选码的定义）

对于给定关系 $R\langle U, F \rangle$ 中的各属性可分为4类：

L类：仅出现在函数依赖左部的属性

R类：仅出现在函数依赖右部的属性

N类：在函数依赖左右两边均未出现的属性

LR类：在函数依赖两边均出现的属性

- (1) 若X是L类属性，则X必为R的任一候选码的成员；
- (2) 若X是L类属性，且 X^+ 包含了R的全部属性，则X必为R的唯一候选码；
- (3) 若X是R类属性，则X必不在任何候选码中；
- (4) 若X是N类属性，则X必包含在R的任一候选码中；
- (5) 若X是LR类属性，且 X^+ 包含了R的全部属性，则X必为R的唯一候选码



【例】 设关系 $R=(A, B, C, D)$; 函数依赖 $F=\{D \rightarrow B, B \rightarrow D, AD \rightarrow B, AC \rightarrow D\}$;

求R的所有候选码

解：

L类属性：A, C；所以AC必是候选码成员，而 $(AC)^+ = ABCD$ ，所以AC是R的唯一候选码



【例】 设关系 $R=(A, B, C, D, E, G)$; 函数依赖 $F=\{AB \rightarrow C, CD \rightarrow E, E \rightarrow A, A \rightarrow G\}$;

求R的所有候选码

解：

- (1) 发现G属于R类属性，所以G一定不是候选码成员；B, D只在左边出现，一定是候选码成员； $(BD)^+ = BD$ ，则对BD进行组合，除了G外，BD可以与ACE进行组合
- (2) $ABD^+ = ABDCEG = U$
- (3) $BDC^+ = BDCEAG = U$
- (4) $BDE^+ = BDEAGC = U$

因为ABD，BDC，BDE的闭包都是ABCDEG，所以R的候选码有三个，分别是ABC，BCD，BDE



【例】设关系模式 $R=\{A, B, C, D, E, G\}$ ，函数依赖集 $F=\{B \rightarrow ADE, A \rightarrow BE, AC \rightarrow G, BC \rightarrow D\}$ ，找出 $r(R)$ 的所有的候选码。

- 因 C 没有在函数依赖的右部出现，故 $X=C$ 为候选码的一部分；
- 因 $C^+=C \neq R$ ，故 C 不是候选码；
- 在函数依赖右部出现但左部不出现的属性有 DEG ，故 $Y=DEG$ ；
- 在 $R-X-Y=AB$ 中寻找与 X 联合起来构成候选码的属性(集)：
 - $(\{A, C\})^+=ACBEGD=R$ ，故 AC 为候选码；
 - $(\{B, C\})^+=BCADEG=R$ ，故 BC 为候选码。
- 因此，关系模式 $r(R)$ 的候选码有 AC 和 BC 。



【定义】如果 $G^+ = F^+$ ，就说函数依赖集 F **覆盖** G （ F 是 G 的覆盖，或 G 是 F 的覆盖），或 F 与 G **等价**。 注 表示能力上是完全相同的

【引理】 $F^+ = G^+$ 的充分必要条件是 $F \subseteq G^+$ ，和 $G \subseteq F^+$

证：必要性显然，只证充分性。

(1) 若 $F \subseteq G^+$ ，则 $X_F^+ \subseteq X_{G^+}^+$ 。

(2) 任取 $X \rightarrow Y \in F^+$ 则有 $Y \subseteq X_F^+ \subseteq X_{G^+}^+$ 。

所以 $X \rightarrow Y \in (G^+)^+ = G^+$ 。即 $F^+ \subseteq G^+$ 。

(3) 同理可证 $G^+ \subseteq F^+$ ，所以 $F^+ = G^+$ 。

如： $F1 = \{sno \rightarrow sname, sno \rightarrow sdept, sdept \rightarrow dname\}$

$F2 = \{sno \rightarrow sname, sdept \rightarrow dname, sno \rightarrow sdept, sno \rightarrow dname\}$

$F1 \equiv F2$



【定义】 如果函数依赖集 F 满足下列条件，则称 F 为一个**极小函数依赖集**。亦称为**最小依赖集**或**最小覆盖**。

- (1) F 中任一函数依赖的右部仅含有一个属性。（**右边**）
- (2) F 中不存在这样的函数依赖 $X \rightarrow A$ ，使得 F 与 $F - \{X \rightarrow A\}$ 等价。（**中间**）
- (3) F 中不存在这样的函数依赖 $X \rightarrow A$ ， X 有真子集 Z 使得 $F - \{X \rightarrow A\} \cup \{Z \rightarrow A\}$ 与 F 等价。（**左边**）



【例】关系模式 $S<U, F>$, 其中 :

$U = \{ \text{Sno}, \text{Sdept}, \text{Mname}, \text{Cno}, \text{Grade} \} ,$

$F = \{ \text{Sno} \rightarrow \text{Sdept}, \text{Sdept} \rightarrow \text{Mname}, (\text{Sno}, \text{Cno}) \rightarrow \text{Grade} \}$

设 $F' = \{ \text{Sno} \rightarrow \text{Sdept}, \text{Sno} \rightarrow \text{Mname}, \text{Sdept} \rightarrow \text{Mname},$

$(\text{Sno}, \text{Cno}) \rightarrow \text{Grade}, (\text{Sno}, \text{Sdept}) \rightarrow \text{Sdept} \}$

F 是最小覆盖, 而 F' 不是。

因为 : $F' - \{ \text{Sno} \rightarrow \text{Mname} \}$ 与 F' 等价

$F' - \{ (\text{Sno}, \text{Sdept}) \rightarrow \text{Sdept} \}$ 也与 F' 等价



【定理6.3】 每一个函数依赖集 F 均等价于一个极小函数依赖集 F_m 。此 F_m 称为 F 的最小依赖集。

证明: 构造性证明 (略)

找出 F 的一个最小依赖集:

(1) 逐一检查 F 中各函数依赖 $FD_i: X \rightarrow Y$, 若 $Y = A_1 A_2 \dots A_k$, $k > 2$,

则用 $\{ X \rightarrow A_j \mid j=1, 2, \dots, k \}$ 来取代 $X \rightarrow Y$ 。 (分解右边)

(2) 逐一检查 F 中各函数依赖 $FD_i: X \rightarrow A$, 令 $G = F - \{X \rightarrow A\}$,

若 $A \in X_G^+$, 则从 F 中去掉此函数依赖。 (分解中间)

(3) 逐一取出 F 中各函数依赖 $FD_i: X \rightarrow A$, 设 $X = B_1 B_2 \dots B_m$,

逐一考查 B_i ($i=1, 2, \dots, m$), 若 $A \in (X - B_i)_F^+$,

则以 $X - B_i$ 取代 X 。 (分解左边)



【例】 $F = \{A \rightarrow B, B \rightarrow A, B \rightarrow C, A \rightarrow C, C \rightarrow A\}$

F_{m1} 、 F_{m2} 都是 F 的最小依赖集：

$$F_{m1} = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$$

$$F_{m2} = \{A \rightarrow B, B \rightarrow A, A \rightarrow C, C \rightarrow A\}$$

- F 的最小依赖集 F_m 不唯一
- 极小化过程(定理6.3的证明)也是检验 F 是否为极小依赖集的一个算法



【例】 已知关系模式 $R\langle U, F \rangle$, 其中 $U=\{A,B,C,D,E,G\}$; $F = \{B \rightarrow D, DG \rightarrow C, BD \rightarrow E, AG \rightarrow B, ADG \rightarrow BC\}$, 求F的最小依赖集

解：

(1) 利用函数依赖的分解性，将函数依赖集中右边分解为单个属性：

$ADG \rightarrow BC$ 分解成 $ADG \rightarrow B, ADG \rightarrow C$

(2) 筛选多余的函数依赖

去除 $B \rightarrow D$, 得 $B_F^+ = \{B\}$ 不包含 D , $B \rightarrow D$ 不能删除

去除 $DG \rightarrow C$, 得 $DG_F^+ = \{D, G\}$ 不包含 C , $DG \rightarrow C$ 不能删除

去除 $BD \rightarrow E$, 得 $BD_F^+ = \{B, D\}$ 不包含 E , $BD \rightarrow E$ 不能删除

去除 $AG \rightarrow B$, 得 $AG_F^+ = \{A, G\}$ 不包含 B , $AG \rightarrow B$ 不能删除

去除 $ADG \rightarrow B$, 得 $ADG_F^+ = \{A, D, G, C, B, E\}$ 包含 B , $ADG \rightarrow B$ 能删除

去除 $ADG \rightarrow C$, 得 $ADG_F^+ = \{A, D, G, C, B, E\}$ 包含 C , $ADG \rightarrow C$ 能删除



【例】已知关系模式 $R\langle U, F \rangle$, 其中 $U = \{A, B, C, D, E, G\}$; $F = \{B \rightarrow D, DG \rightarrow C, BD \rightarrow E, AG \rightarrow B, ADG \rightarrow BC\}$, 求 F 的最小依赖集

解 :

经过筛选之后 , 函数依赖集 F 变为 :

$$F = \{B \rightarrow D, DG \rightarrow C, BD \rightarrow E, AG \rightarrow B\}$$

(3) 化简函数依赖左边不为单个属性的函数依赖

1. $DG \rightarrow C$

去除 D , $G_F^+ = \{G\}$ 不包含 C , D 不能去除 ; 去除 G , $D_F^+ = \{D\}$ 不包含 C , G 不能去除 ,
 $DG \rightarrow C$ 已是最简

2. $BD \rightarrow E$

去除 B , $D_F^+ = \{D\}$ 不包含 E , B 不能去除 ; 去除 D , $B_F^+ = \{B, D, E\}$ 包含 E , D 可去除 ,
 $BD \rightarrow E$ 可以简化为 $B \rightarrow E$

3. $AG \rightarrow B$, 同理已是最简 **最小依赖集是 $F_m = \{B \rightarrow D, DG \rightarrow C, B \rightarrow E, AG \rightarrow B\}$**



关系数据库理论



6.1 问题的提出

6.2 规范化

6.3 数据依赖的公理系统

6.4 模式的分解



- 把低一级的关系模式分解为若干个高一级的关系模式的方法不是唯一的；
- 只有能够保证分解后的关系模式与原关系模式等价，分解方法才有意义。

目的: 规范化的目的是使结构合理，使数据冗余尽量小，清除插入，删除和更新异常。

方法: 将关系模式投影分解成两个或两个以上的关系模式，但关系模式的分解不是唯一的。

要求: 分解后的关系模式集合应当与原关系模式“等价”。

一个关系模式达到BCNF，说明在函数依赖的范畴内，已实现了彻底分离，可消除“异常”，但在实际应用中，并不一定要求全部模式都达到BCNF。



【定义】 关系模式 $R\langle U, F \rangle$ 的一个分解：

$$\rho = \{ R_1\langle U_1, F_1 \rangle, R_2\langle U_2, F_2 \rangle, \dots, R_n\langle U_n, F_n \rangle \}$$

其中 $U = U_1 \cup U_2 \cup \dots \cup U_n$ ，且不存在 $U_i \subseteq U_j, i \neq j$, F_i 为 F 在 U_i 上的投影

【定义】 函数依赖集合 $\{X \rightarrow Y \mid X \rightarrow Y \in F^+ \wedge XY \subseteq U_i\}$ 的一个覆盖 F_i 叫作 F 在属性 U_i 上的投影



- 将一个关系模式 $R\langle U, F \rangle$ 分解为若干个关系模式 $R_1\langle U_1, F_1 \rangle, R_2\langle U_2, F_2 \rangle, \dots, R_n\langle U_n, F_n \rangle$, 意味着将存储在一张二维表 r 中的数据分散到了若干二维表 $r_1, r_2, \dots, r_n$ 中。
- 这样的分解不应该丢失信息, 即通过对关系 $r_1, r_2, \dots, r_n$ 的自然连接能重新得到关系 r 中的所有信息。
- 事实上, 将关系 r 投影为 $r_1, r_2, \dots, r_n$ 时不会丢失信息, 关键是对 $r_1, r_2, \dots, r_n$ 做自然连接时可能产生一些 r 中原来没有的元组, 从而无法区别哪些元组是 r 中原来有的, 哪些是不应该有的。在这个意义上就丢失了信息。



对于一个分解是多种多样的，但分解后产生的模式应与原模式等价，对等价的概念形成三种不同的解释：

1. 分解具有无损连接性
2. 分解要保持函数依赖
3. 分解既要**保持函数依赖**，又要具有**无损连接性**



关系模式 $R\langle U, F \rangle$ 的一个分解 $\rho = \{ R_1\langle U_1, F_1 \rangle, R_2\langle U_2, F_2 \rangle, \dots, R_n\langle U_n, F_n \rangle \}$

若 R 与 R_1 、 R_2 、...、 R_n 自然连接的结果相等，则称关系模式 R 的这个**分解 ρ 具有无损连接性 (Lossless join)**

- 具有无损连接性的分解保证不丢失信息
- 无损连接性不一定能解决插入异常、删除异常、修改复杂、数据冗余等问题



设关系模式 $R\langle U, F \rangle$ 被分解为若干个关系模式

$R_1\langle U_1, F_1 \rangle$, $R_2\langle U_2, F_2 \rangle$, ... , $R_n\langle U_n, F_n \rangle$

(其中 $U = U_1 \cup U_2 \cup \dots \cup U_n$, 且不存在 $U_i \subseteq U_j$, $i \neq j$, F_i 为 F 在 U_i 上的投影) , 若 F

所逻辑蕴含的函数依赖一定也由分解得到的某个关系模式中的函数依赖 F_i 所逻辑

蕴含 , 则称关系模式 R 的这个分解是保持函数依赖的 (Preserve
dependency)



S-D-L (Sno , Dept , Loc) 假设每个系的学生住在同一个地方

- 三种分解方案：

方案1：S-L (Sno , Loc) , D-L (Dept , Loc)

方案2：S-D (Sno , Dept) , S-L (Sno , Loc)

方案3：S-D (Sno , Dept) , D-L (Dept , Loc)



这三种分解方案是否都满足分解要求呢？

假设此关系模式的数据如表所示，此关系用 r 表示。

Sno	Dept	Loc
S01	D1	L1
S02	D2	L2
S03	D2	L2
S04	D3	L1



若按方案1将S-D-L投影到S-L和D-L的属性上，然后做自然连接。

Sno	Loc
S01	L1
S02	L2
S03	L2
S04	L1

Dept	Loc
D1	L1
D2	L2
D3	L1

自然连接

Sno	Dept	Loc
S01	D1	L1
S01	D3	L1
S02	D2	L2
S03	D2	L2
S04	D1	L1
S04	D3	L1



- 将关系模式 $R\langle U, F \rangle$ 分解为个关系模式 $R_1\langle U_1, F_1 \rangle, R_2\langle U_2, F_2 \rangle, \dots, R_n\langle U_n, F_n \rangle$, 若对于 R 中的任何一个可能的 r , 都有 $r = r_1 * r_2 * \dots * r_n$, 即 r 在 $R_1, R_2, \dots, R_n$ 上的投影的自然连接等于 r , 则称关系模式 R 的这个分解具有**无损连接性**。
- 方案1不具有无损连接性。



分析方案2。将S-D-L投影到S-D，S-L的属性上，然后做自然连接。

Sno	Dept
S01	D1
S02	D2
S03	D2
S04	D3

Sno	Loc
S01	L1
S02	L2
S03	L2
S04	L1

自然连接

Sno	Dept	Loc
S01	D1	L1
S02	D2	L2
S03	D2	L2
S04	D3	L1



如果 (S03,D2)
改为 (S03,D3)

Sno	Dept
S01	D1
S02	D3
<u>S03</u>	<u>D3</u>
S04	D3

则 (S03,L2) 需
改为 (S03,L1)

Sno	Loc
S01	L1
S02	L2
<u>S03</u>	<u>L1</u>
S04	L1

Sno	Dept	Loc
S01	D1	L1
S02	D2	L2
S03	D2	L2
S04	D3	L1

如果这两个修改没有同时进行，则会出现不一致信息。
因此方案2没有保持原有的函数依赖关系：

Dept→Loc



- 分解方案3，将S-D-L投影到S-D，D-L的属性上，既满足无损连接性，又保持了原有的函数依赖关系，因此是好的分解方法。

Sno	Dept
S01	D1
S02	D2
S03	D2
S04	D3

Dept	Loc
D1	L1
D2	L2
D3	L3

- 分解具有无损连接性和分解保持函数依赖是两个独立的标准。具有无损连接性的分解不一定保持函数依赖；保持函数依赖的分解不一定具有无损连接性。



- 一般情况下，在进行模式分解时，应将**有直接依赖关系的属性放置在一个关系模式中**，
- 这样得到的分解结果一般具有无损连接性，并能保持函数依赖关系不变。



【算法】判别一个分解的无损连接性

$\rho = \{R_1(U_1, F_1), R_2(U_2, F_2), \dots, R_k(U_k, F_k)\}; U = (A_1, \dots, A_n)$

(1) 建立一张n行k列的表，每一列对应一个属性，每一行对应一个分解后的关系模式，若属性 $A_j \in U_i$ ，则在i行j列填上 a_j ，否则填 b_{ij}

(2) 遍历F中的每一个函数依赖 $X \rightarrow Y$ ，在表中寻找X所有分量列中符号全相同的行，若找到，则在这些行对应的Y属性上置相同符号，否则改为 b_{mj} ，m为最小行号

(3) 修改后如果有一行变为 $a_1, a_2, \dots, a_n$ ，则 ρ 是无损连接分解



【例】已知 $R\langle U, F \rangle$, $U=(A, B, C, D, E)$; $F=\{AB\rightarrow C, C\rightarrow D, D\rightarrow E\}$, R 的一个分解 $\rho=\{R_1(A, B, C), R_2(C, D), R_3(D, E)\}$ 判断无损连接性

(1) 建立一张n行k列的表

	A	B	C	D	E
R_1	a_1	a_2	a_3	b_{14}	b_{15}
R_2	b_{21}	b_{22}	a_3	a_4	b_{25}
R_3	b_{31}	b_{32}	b_{33}	a_4	a_5



【例】已知 $R \langle U, F \rangle$, $U = (A, B, C, D, E)$; $F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow E\}$, R 的一个分解 $\rho = \{R_1(A, B, C), R_2(C, D), R_3(D, E)\}$ 判断无损连接性

(2) 遍历 F 中的每一个函数依赖 $X \rightarrow Y$,在表中寻找所有分量列中符号全相同的行 , 若找到 , 则在这些行对应的 Y 属性上置相同符号 , 否则改为 b_{mj} , m 为最小行号

$AB \rightarrow C$	A	B	C	D	E
R_1	a_1	a_2	a_3	b_{14}	b_{15}
R_2	b_{21}	b_{22}	a_3	a_4	b_{25}
R_3	b_{31}	b_{32}	b_{33}	a_4	a_5



【例】已知 $R \langle U, F \rangle$, $U = (A, B, C, D, E)$; $F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow E\}$, R 的一个分解 $\rho = \{R_1(A, B, C), R_2(C, D), R_3(D, E)\}$ 判断无损连接性

(2) 遍历 F 中的每一个函数依赖 $X \rightarrow Y$,在表中寻找所有分量列中符号全相同的行 , 若找到 , 则在这些行对应的 Y 属性上置相同符号 , 否则改为 b_{mj} , m 为最小行号

$C \rightarrow D$	A	B	C	D	E
R_1	a_1	a_2	a_3	a_4	b_{15}
R_2	b_{21}	b_{22}	a_3	a_4	b_{25}
R_3	b_{31}	b_{32}	b_{33}	a_4	a_5



【例】已知 $R \langle U, F \rangle$, $U = (A, B, C, D, E)$; $F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow E\}$, R 的一个分解 $\rho = \{R_1(A, B, C), R_2(C, D), R_3(D, E)\}$ 判断无损连接性

(2) 遍历 F 中的每一个函数依赖 $X \rightarrow Y$,在表中寻找所有分量列中符号全相同的行 , 若找到 , 则在这些行对应的 Y 属性上置相同符号 , 否则改为 b_{mj} , m 为最小行号

$D \rightarrow E$	A	B	C	D	E
R_1	a_1	a_2	a_3	a_4	a_5
R_2	b_{21}	b_{22}	a_3	a_4	a_5
R_3	b_{31}	b_{32}	b_{33}	a_4	a_5

(3) 修改后如果有一行变为 $a_1, a_2, \dots, a_n$, 则 ρ 是**无损连接分解** ✓



【算法】（合成法）转换成3NF保持函数依赖的分解

- (1) 对 $R\langle U, F \rangle$ 中的依赖集 F 求最小依赖集 F_m
- (2) 把 R 中没有出现在 F_m 中的属性去掉并单独组成一个模式
- (3) 对任意 $X \rightarrow A \in F_m$ ，则以 XA 组成一个模式；若 $X \rightarrow A_1, X \rightarrow A_2 \dots X \rightarrow A_m$ 都属于 F_m ，则以 $XA_1A_2 \dots A_m$ 组成一个模式（即将 m 个模式合并成一个模式）



找出 F 的一个最小依赖集：

(1)逐一检查 F 中各函数依赖 $FD_i: X \rightarrow Y$, 若 $Y = A_1 A_2 \dots A_k$, $k > 2$,

则用 $\{ X \rightarrow A_j | j=1, 2, \dots, k \}$ 来取代 $X \rightarrow Y$ 。(分解右边)

(2)逐一检查 F 中各函数依赖 $FD_i: X \rightarrow A$, 令 $G = F - \{X \rightarrow A\}$,

若 $A \in X_G^+$, 则从 F 中去掉此函数依赖。(分解中间)

(3)逐一取出 F 中各函数依赖 $FD_i: X \rightarrow A$, 设 $X = B_1 B_2 \dots B_m$,

逐一考查 B_i ($i=1, 2, \dots, m$), 若 $A \in (X - B_i)_F^+$,

则以 $X - B_i$ 取代 X 。(分解左边)



【例】 已知 $R\langle U, F \rangle$, $U=(A,B,C,D,E)$; $F=\{A\rightarrow BC, ABD\rightarrow CE, E\rightarrow D\}$ 分解为3NF并保持函数依赖

(1) 计算F的最小依赖集 $F_m=\{A\rightarrow B, A\rightarrow C, AD\rightarrow E, E\rightarrow D\}$

(2) U中没有属性不出现在 F_m 中, 跳过

(3) X为A时, $A\rightarrow B, A\rightarrow C$ 分为一组; X为AD时, $AD\rightarrow E$ 一组; X为E时, $E\rightarrow D$ 一组。最终分解结果 $R_1(A,B,C), R_2(A,D,E), R_3(E,D)$



【算法】转换为3NF既有无损连接性又保持函数依赖的分解

- (1) 用合成法转换成3NF保持函数依赖**
- (2) 求出Fm的候选码X**
- (3) 将候选码单独组成模式，然后将其与 (1) 的结果取并集**
- (4) 观察新组成的分解模式是否有包含关系，有就去掉**



【例】已知 $R\langle U, F \rangle$, $U=(A, B, C, D, E)$; $F=\{A \rightarrow BC, ABD \rightarrow CE, E \rightarrow D\}$ 分解为3NF并保持无损连接和函数依赖

(1) 用合成法得到 $R_1(A, B, C)$, $R_2(A, D, E)$, $R_3(E, D)$

(2) 计算Fm候选码为AD和AE

(3) 将候选码单独组成模式 $R_4(A, B)$, $R_5(A, E)$, 与 (1) 的结果取并集得到 $R_1(A, B, C)$, $R_2(A, D, E)$, $R_3(E, D)$, $R_4(A, B)$, $R_5(A, E)$

(4) 删除包含关系的模式得到结果 $R_1(A, B, C)$, $R_2(A, D, E)$



【算法】（分解法）转换为BCNF的无损连接分解

(1) 令 $\rho = R(U, F)$

(2) 检查 ρ 中所有模式是否都是BCNF，若是，算法终止

(3) 设 ρ 中的模式 $R_i(U_i, F_i)$ 不属于BCNF，那么必有 $X \rightarrow A$ 且 X 不是 R_i 的码。对 R_i 进行分解成两个模式： $U_{s1} = (XA)$, $U_{s2} = U_i - (A)$ ，并计算 F 在 U_{s1} 和 U_{s2} 的依赖投影

(4) 返回第(2)步直到终止



【例】已知 $R\langle U, F \rangle$, $U=(A, B, C, D, E)$; $F=\{A \rightarrow C, C \rightarrow D, B \rightarrow C, BD \rightarrow C, CE \rightarrow A\}$ 分解为BCNF并保持无损连接

(1) 令 $\rho=R(U, F)$

(2) ρ 中不是所有模式都是BCNF , 进下一步

(3) R 的码是 BE , 那么 $A \rightarrow C$ 依赖关系中 A 不包含码 , 分解成 $R_1=(AC)$, $R_2=(ABDE)$, 并计算 F_1 和 F_2

(4) R_2 不满足BCNF , 分解成 $R_{21}=(BD)$, $R_{22}=(ABE)$, 并计算 F_{21} 和 F_{22}

(5) R_1, R_{21}, R_{22} 都满足BCNF , 算法结束 , 分解结果为 $R_1=(AC)$, $R_{21}=(BD)$, $R_{22}=(ABE)$



小结

- **函数依赖**

 - 平凡函数依赖与非平凡函数依赖

 - 完全函数依赖与部分函数依赖

 - 传递函数依赖

 - 多值依赖

- **范式的概念**

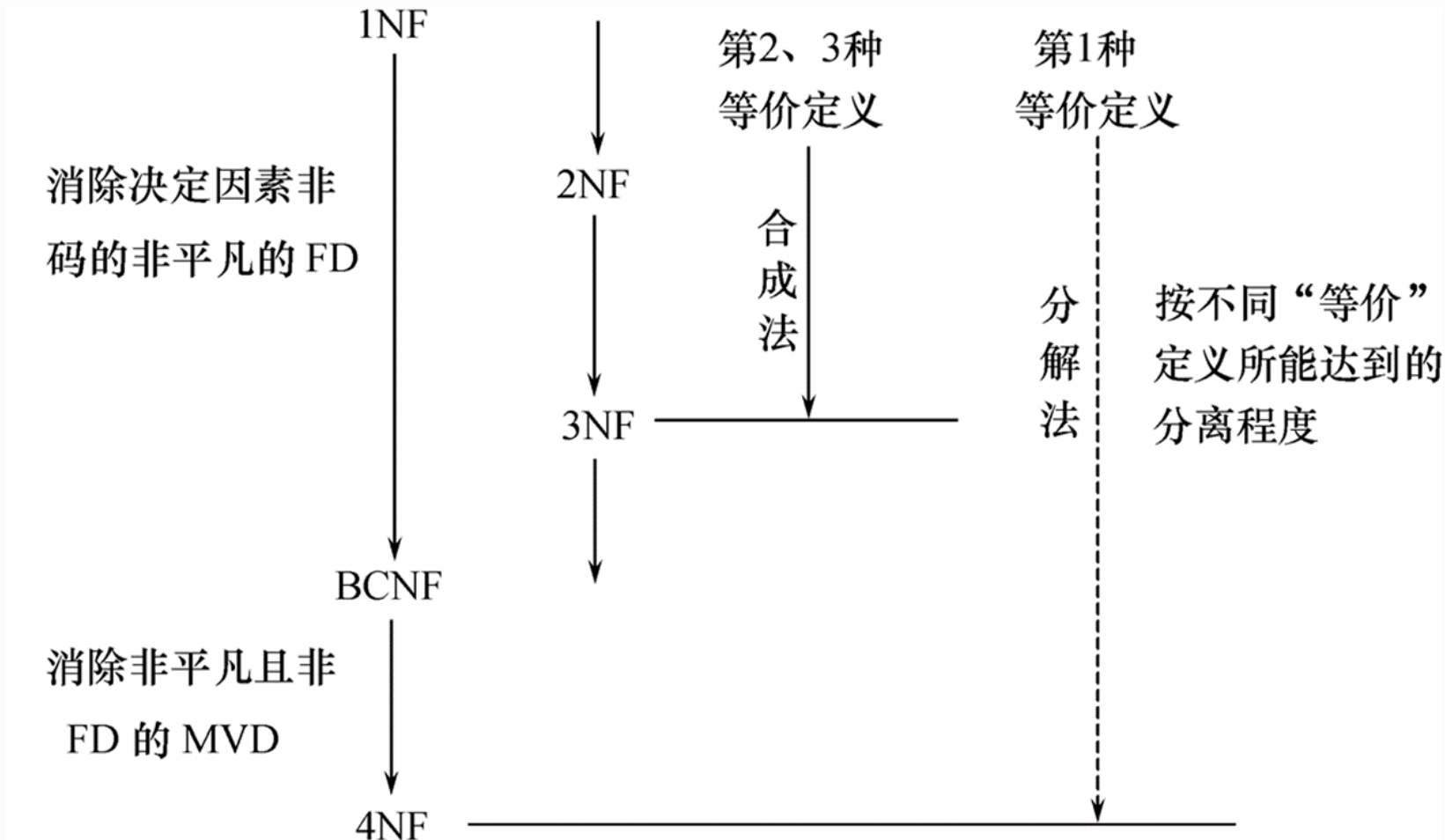
- **关系模式规范化**

- **Armstrong公理系统**

- **模式的分解**



小结



FD是函数依赖，MVD是多值函数依赖



小结

Armstrong公理系统：

- 自反律；增广律；传递律
- 合并规则；伪传递规则；分解规则
- 函数依赖闭包 $F +$ 求属性集 X 关于 F 的闭包 XF
- 最小依赖集



小结

关系模式的规范化基本思想

1. 逐步消除不合适的数据依赖中使模式中的各关系模式达到某种程度的“分离”——模式分解
2. 一事一地的模式设计原则
3. 规范化理论为数据库设计提供了理论指南和算法工具。
4. 并不是规范化程度越高，模式就越好，结合应用环境的具体情况，合理地设计数据库模式。